KAVYA TRANSPORTS ERP

Banking & Payment Automation

Master Implementation Prompt

FastAPI • React/TSX • Flutter • PostgreSQL • MongoDB • Redis • AWS

⚡ **How to use this prompt**

Copy the MASTER PROMPT section (Part 3) in full and paste it directly into GitHub Copilot Chat, Claude, or GPT-4o. It is self-contained — all context your AI needs is embedded. Use it section by section or all at once. Each sub-prompt can also be used independently.

Part 1 — Project Context & Tech Stack

This document defines the complete implementation scope for the Banking & Payment Automation module of the Kavya Transports ERP system. All code generated from the master prompt must conform strictly to this context.

1.1 Tech Stack (Non-Negotiable)

Layer	Technology	Notes
Backend API	FastAPI (Python 3.11+)	30+ routers, 200+ endpoints pattern
Frontend	React 18 + TypeScript	TanStack Query, Zustand, Tailwind CSS
Mobile App	Flutter (Android)	Foreground service, Hive offline, Dio
Primary DB	PostgreSQL 15	Main transactional store
Document DB	MongoDB	Event logs, audit trail
Cache	Redis	Sessions, rate limit, 24h API cache
Task Queue	Celery + Redis broker	All background/scheduled jobs
File Storage	AWS S3 (boto3)	ePOD, invoices, statements
Email	AWS SES	Transactional + digest emails
Push	Firebase FCM	Mobile push notifications
Payments	Razorpay	Payment links + webhooks
Banking	Account Aggregator (Finvu/Setu)	Bank statement import
WhatsApp	Twilio / Gupshup WABA	Business messaging
Toll	NPCI NETC FASTag API	Daily toll auto-log

1.2 Existing Architecture Patterns

All generated code must follow these patterns already established in the codebase:

- Router files live in app/routers/{module}.py — register in main.py
- Schemas use Pydantic v2 with model_config = ConfigDict(from_attributes=True)
- Database sessions injected via Depends(get_db) — never create sessions manually
- All background tasks use Celery — never asyncio.create_task for long-running work
- Redis cache wrapper: await cache.get(key) / await cache.set(key, val, ttl=86400)
- MongoDB audit log: await mongo.event_log.insert_one({event, entity, data, ts})
- Role guard decorator: @require_roles(['accountant', 'admin']) on every endpoint
- Frontend API calls: useMutation / useQuery from TanStack Query — no raw fetch
- Zustand store slices in store/{module}Store.ts — no prop drilling
- All amounts stored as INTEGER paise (₹1 = 100) — never float

1.3 9-Role RBAC Reference

Role	Relevant Permissions for This Module
super_admin	Full access — all banking, payment, ledger, reports
admin	Same as super_admin minus system config
accountant	Primary role — invoices, payments, banking, ledger, GST
fleet_manager	View only — payment status on trips
driver	View own settlement status only
client	View own invoices + pay via link — no internal access
auditor	Read-only — all finance + GSTR export
pump_operator	Fuel expenses only
branch_manager	Branch-scoped finance view

Part 2 — Full Automation Scope

The following 7 automation groups must be implemented. Each maps to a sidebar module in the ERP.

💳 Group 1 — Invoice Automation (Finance › Invoices)

#	Automation	Trigger	Output
INV-01	Auto-create invoice on trip delivery	Trip status → DELIVERED	Invoice row in PostgreSQL, PDF in S3
INV-02	Auto-dispatch invoice to client	Invoice created	WhatsApp + AWS SES with PDF link
INV-03	Double-billing detection	Before any invoice creation	Block + alert if duplicate LR/trip found
INV-04	Freight leakage flag	Invoice amount vs rate card	Flag if deviation > 5%, alert accountant
INV-05	Partial payment tracking	Razorpay partial capture webhook	Outstanding balance updated, status = partial
INV-06	E-Way Bill auto-attach	Invoice creation for interstate	EWB number fetched/generated, attached to invoice

💰 Group 2 — Payment Automation (Finance › Payments)

#	Automation	Trigger	Output
PAY-01	Razorpay payment link auto-send	Invoice created	Link via WhatsApp + email to client
PAY-02	Webhook reconciliation	payment.captured from Razorpay	Invoice paid, ledger entry, receipt sent
PAY-03	Payment failure retry alert	payment.failed webhook	Alert to client + accountant, retry link
PAY-04	Refund auto-initiation	Credit note approved by admin	Razorpay refund API called, ledger reversed
PAY-05	Payment receipt auto-send	Payment confirmed	PDF receipt via WhatsApp + email to client
PAY-06	Quick Payment Link tool	Accountant uses Tools > Payment Link	One-time Razorpay link for ad-hoc payments

🏧 Group 3 — Banking Automation (Accountant › Banking)

#	Automation	Trigger	Output
BNK-01	Bank statement auto-import	Daily cron 6 AM / manual trigger	Transactions in bank_transactions table
BNK-02	Auto-match transactions to invoices	Post-import cron	Matched = reconciled, unmatched = flagged
BNK-03	Exception workflow	Unmatched transaction	Alert to accountant, exception queue
BNK-04	Manual match UI	Accountant resolves exception	Manual link saved, removed from exception queue
BNK-05	FASTag toll auto-log	NETC API daily pull 9 PM	Toll entries linked to vehicle + trip
BNK-06	Banking entry CRUD	Accountant creates manual entry	Journal entry in ledger, bank balance updated

📒 Group 4 — Ledger Automation (Accountant › Ledger)

#	Automation	Trigger	Output
LDG-01	Auto ledger posting on payment	Razorpay webhook confirmed	Dr/Cr entry, account head auto-classified
LDG-02	Receivable auto-close	Full payment received	Receivable entry status = closed
LDG-03	Client ledger real-time sync	Any transaction affecting client	client_ledger table updated, balance recalculated
LDG-04	Expense auto-post	Expense approved by manager	Ledger Dr entry under correct account head
LDG-05	Contra entry auto-generate	Cash transferred between accounts	Contra journal entry with both legs

📤 Group 5 — Receivables & Payables (Accountant module)

#	Automation	Trigger	Output
REC-01	Overdue detection cron	Daily 7 AM cron	Invoices past due date → status = overdue
REC-02	Payment reminder sequence	30 / 15 / 7 days before due	WhatsApp + email chain to client
REC-03	Overdue escalation	Invoice overdue > 7 days	Alert to admin + manager with aging summary
PAB-01	Driver settlement auto-calc	Trip complete + expenses submitted	Net payable computed, advance deducted
PAB-02	Supplier payable auto-create	Market trip hired vehicle used	Payable against supplier, linked to trip
PAB-03	Payable due date alert	3 days before payable due	WhatsApp + in-app alert to accountant

🚨 Group 6 — Alerts (Tools › Notifications)

#	Automation	Trigger	Output
ALT-01	Low wallet balance alert	Company balance < ₹5,000	FCM push + email to finance manager
ALT-02	Large payment alert	Payment > ₹1,00,000 received	Instant notification to admin
ALT-03	Failed bank import alert	Account Aggregator API error	Alert to accountant + retry scheduled
ALT-04	GST filing reminder	20th of every month cron	GSTR-1 due reminder to accountant
ALT-05	Audit access expiry alert	Auditor session expiring in 1 hour	Alert to auditor + auto-revoke on expiry

📊 Group 7 — Reports (Accountant › Reports)

#	Automation	Trigger	Output
RPT-01	Daily finance digest	Cron 9 PM daily	Collections, outstanding, expenses — email to management
RPT-02	Weekly P&L auto-report	Cron Monday 7 AM	Vehicle/route/client P&L — emailed as PDF
RPT-03	Monthly close report	Cron 1st of month 8 AM	Full reconciliation summary + aging — email + S3
RPT-04	GSTR-1 Excel auto-export	Monthly cron or on-demand	GST-compliant Excel for auditor module
RPT-05	Receivables aging report	On-demand + weekly	Bucket aging: 0-30, 31-60, 61-90, 90+ days

Part 3 — MASTER PROMPT (Copy This Entirely)

📋 **Instructions**

Select everything from the dashed line below to the end of this document and paste it into your AI coding assistant. It is designed to work with GitHub Copilot Chat, Claude, or GPT-4o in one shot.

MASTER PROMPT START

You are a senior full-stack engineer implementing the Banking & Payment Automation module for Kavya Transports ERP. Implement everything below **in full — no placeholders, no TODOs, no stubs.**

TECH STACK — FOLLOW EXACTLY, NO SUBSTITUTIONS

```
Backend  : FastAPI (Python 3.11), Pydantic v2, SQLAlchemy 2.0 async
Frontend : React 18 + TypeScript, TanStack Query v5, Zustand, Tailwind
Mobile   : Flutter 3.x (Android), Dio, Hive, flutter_background_service
Database : PostgreSQL 15 (primary), MongoDB (audit/events), Redis (cache)
Jobs     : Celery + Redis broker (APScheduler for cron triggers)
Storage  : AWS S3 via boto3, AWS SES for email
Push     : Firebase FCM (mobile), WhatsApp via Twilio WABA
Payments : Razorpay (payment links + webhooks)
Banking  : Account Aggregator API (Finvu/Setu)
Toll     : NPCI NETC FASTag API
GST      : NIC E-Way Bill API + GST Verification API
```

CODING RULES — MUST BE FOLLOWED IN EVERY FILE

1. All money amounts stored as INTEGER PAISE (₹1 = 100). Never float.
2. All DB sessions via Depends(get_db). Never create sessions directly.
3. All background work via Celery tasks. Never asyncio.create_task.
4. Every endpoint has @require_roles([...]) guard — see RBAC below.
5. Cache with await cache.get/set(key, val, ttl). TTL=86400 for API data.
6. Every write operation logs to MongoDB: {event, entity_id, data, user, ts}.
7. External API calls wrapped in Tenacity: 3 retries, exponential backoff.
8. Frontend: useQuery/useMutation only. No raw fetch or axios.
9. Zustand stores in store/{module}Store.ts. No prop drilling.
10. Pydantic schemas: model_config = ConfigDict(from_attributes=True).

Part 4 — Individual Sub-Prompts

Use these when you want to implement one specific section at a time rather than everything at once. Each is self-contained.

Sub-Prompt A — Invoice Auto-Generation Only

Paste this:

Implement app/routers/invoices.py and app/tasks/finance_tasks.py (send_invoice_to_client task only) for Kavya Transports ERP. Stack: FastAPI + PostgreSQL + Razorpay + AWS SES + Twilio WABA. Invoice is auto-created when trip status changes to DELIVERED. Amount in paise (integer). Generate invoice_no as KT-{YYYY}-{MM}-{seq}. Create Razorpay order. Generate PDF via WeasyPrint, upload to S3. Send via WhatsApp + email. Log to MongoDB. RBAC: accountant+. No stubs.

Sub-Prompt B — Razorpay Webhook Only

Paste this:

Implement POST /payments/webhook/razorpay for Kavya Transports ERP. Stack: FastAPI + Razorpay + PostgreSQL + Redis. Verify hmac_sha256 signature. Handle payment.captured (mark invoice paid, post ledger Dr/Cr, send receipt via SES+WA), payment.failed (alert client+accountant via FCM), refund.processed (reverse ledger). All processing in Celery task — return 200 immediately. Idempotency check on razorpay_payment_id. Amounts in paise. No stubs.

Sub-Prompt C — Banking Reconciliation Only

Paste this:

Implement app/routers/banking.py and auto_match_transactions Celery task for Kavya Transports ERP. Stack: FastAPI + PostgreSQL + Account Aggregator API (Finvu) + Redis. Import bank statements daily. Auto-match credit transactions to invoices: match on amount_paise + date ±7 days. 1 match = reconcile. 0 or 2+ = exception, alert accountant via FCM. Expose exception queue endpoint. Manual match endpoint. RBAC: accountant+. No stubs.

Sub-Prompt D — Frontend Receivables Page Only

Paste this:

Implement src/pages/accountant/ReceivablesPage.tsx for Kavya Transports ERP. Stack: React 18 + TypeScript + TanStack Query v5 + Zustand + Tailwind + Recharts. Show aging buckets as KPI cards (0-30, 31-60, 61-90, 90+ days). Drill-down table per bucket with send reminder action. Bulk reminder button. Bar chart using Recharts. API: GET /receivables/aging + GET /receivables/. Role guard: accountant, admin. No stubs, no TODOs.

Sub-Prompt E — Celery Cron Jobs Only

Paste this:

Implement app/tasks/cron_schedule.py and all scheduled Celery tasks for Kavya Transports ERP finance module. Stack: Celery + Redis + APScheduler. Jobs: 6AM bank import, 7AM overdue detection, 8AM payable due alerts, 9PM FASTag pull, 9PM daily digest (AWS SES), Monday 7AM weekly P&L PDF, 1st of month close report, rolling 30/15/7 day invoice reminders (WhatsApp+SES), 20th month GST reminder. Full implementation only.

Part 5 — Required Environment Variables

Variable	Where to get it	Required for
RAZORPAY_KEY_ID	Razorpay Dashboard → API Keys	Payment links + orders
RAZORPAY_KEY_SECRET	Razorpay Dashboard → API Keys	Payment API calls
RAZORPAY_WEBHOOK_SECRET	Razorpay Dashboard → Webhooks	Webhook signature verify
AA_CLIENT_ID	Finvu / Setu AA sandbox registration	Bank statement import
AA_CLIENT_SECRET	Finvu / Setu AA sandbox registration	Bank statement import
AA_BASE_URL	https://finvu.in/api/v1 (sandbox/prod)	Bank statement import
NETC_API_KEY	NPCI NETC partner registration	FASTag toll fetch
NETC_BASE_URL	NPCI NETC portal	FASTag toll fetch
AWS_SES_REGION	Your AWS region (e.g. ap-south-1)	Email sending
SES_FROM_EMAIL	Verified SES sender address	Email sending
TWILIO_ACCOUNT_SID	Twilio Console	WhatsApp messages
TWILIO_AUTH_TOKEN	Twilio Console	WhatsApp messages

Variable	Where to get it	Required for
TWILIO_WHATSAPP_FROM	whatsapp:+14155238886 (sandbox)	WhatsApp messages
FCM_SERVER_KEY	Firebase Console → Project Settings	Mobile push
S3_BUCKET_NAME	Your AWS S3 bucket	PDF + file storage
CELERY_BROKER_URL	redis://localhost:6379/0	Background tasks

Security Note

Never commit these to Git. Use .env locally and AWS Secrets Manager / Parameter Store in production. The FastAPI app should use python-dotenv locally and boto3 SSM in production.